

PhyriadFG — Estructura nueva + dependencias y conexiones

Estructura POST-separación (cli / core / capture / flow / warp_blend / present /
instrument + framework + shaders)

Nota

Esta es la estructura **POST-separación**. El *code-map* viejo (PHYRIADFG_CODEMAP_SLIDES) describe el **MONOLITO anterior** (main.cpp único) y **se conserva**. Mapa para navegar/tunear el código sin leerlo todo: módulos, DAG de dependencias, flujo C/F/P, y ripples por elemento.

Mapa de módulos (árbol nuevo)

src/

cli/ — Config + parse_args/print_help (parser CLI, hoja; solo stdlib)

core/ — FgContext + VDev/Img/HBuf + globals + main() (substrato + orquestador)

capture/ — run_capture (Thread C): D3D11 WGC/DDA → convert/pack

flow/ — run_flow (Thread F): flujo óptico + GME + holón → publica pares

warp_blend/ — WapPipe/FieldPipe/FillPipe (fábricas warp/blend; sin FgContext)

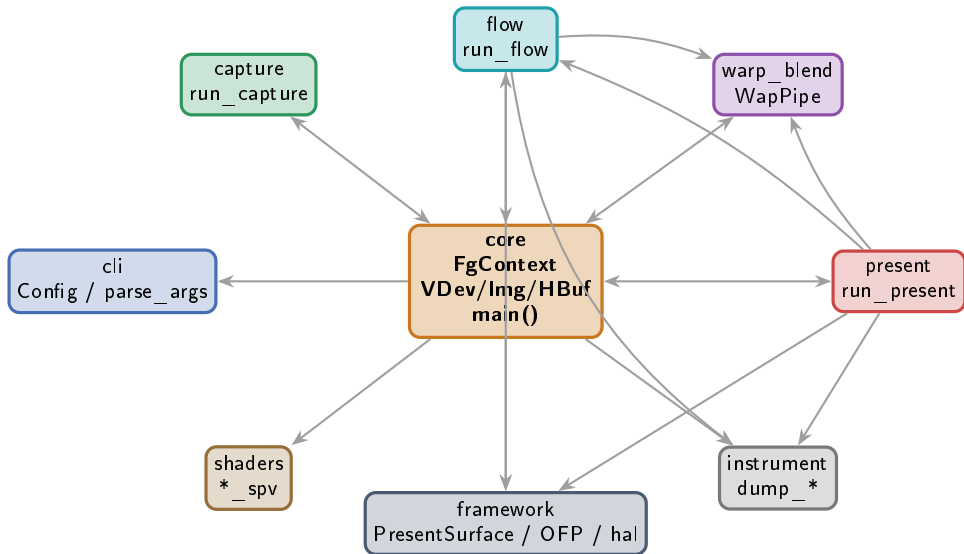
present/ — run_present (Thread P): PresentSurface + warp/upscale/blit/present

instrument/ — dump_bmp / dump_rgba (volcado de frames)

framework/ **framework** — PresentSurface, OpticalFlowPipeline, hal, topology,
schema (substrato vendido)

shaders/ **shaders** — *_spv.hpp SPIR-V embebido (hdr_convert, wap_warp, gme_*,
mv_*, upscale_*, ...)

DAG de dependencias (depends_on / #includes)



Flujo de datos C / F / P (per-frame, izquierda→derecha)

FgContext — estado compartido (refs a locales de main())

run_capture (Thread C)

WGC ring-map / DXGI AcquireNextFrame → Astage[s] (s=c_seq%cap_slots) → convert/pack (iGPU cpPipe / A cvPipe → hR_a[s]) → stamp c_slots[s].t_cap_ms

c_seq.fetch_add
+ c_cv ↓
anillo: c_slots[s],
Astage, hR_a[s]

run_flow (Thread F)

wait c_cv → flujo óptico A↔B (ofp/ofpA / nvofa) → GME afín + máscara dissidence (gme_fit_affine / gme_record) → MV smooth/median + holón (object_repair) → escribe f_pair_*[gen]

f_seq.fetch_add
+ f_cv ↓
f_pair_*[gen] +
hl_a/hMV_a/hSAD_a/hDIS_a

↑ p_presenting ↑
(guarda de anillo /
back-pressure)

run_present (Thread P)

tick pautado (paced_wait_P) → sube MV/DIS a wap*A → wap_warp fase t → fill/matte/median/bg-snap → blit → bridge_img (A.q) → PresentSurface.submit

Panel / DComp (salida pautada)

Tabla por módulo I

Módulo	Provee clave)	(símbolos	Depende de	Campos FgContext	Rol
cli	Config; CaptureApi/FgGpu/ConvPackPipe/UnpackPipe; kMaxInterp/kAutoNMax; print_help, parse_args, parse_extra	enums solo	stdlib ninguno		Hoja CLI: define Config + parser; no toca FgContext/threads.
core	FgContext, RealSlot, VDev, Img, HBuf; vdev_create, img_create, hbuf_import, submit_wait(_q2), oneshot, route_for, vk_live, globals (g_quit...), main()	cli, capture, flow, warp_blend, present, instrument, framework, shaders	define TODOS (c_seq, f_seq, p_presenting, f_pair_*, c_slots, pacing)		Substrato + orquestador: ctx + spawn/join de los 3 threads.
capture	run_capture; D3D, OutInfo, ConvPackPipe, UnpackPipe; d3d_init/shutdown/staging(_on),	core	(de-vice/vk_util/fg_context)ts[s].t_cap_ms, vulkan.h	escribe c_seq, c_cv, Thread C: lee cfg, d, wgc_ctx, Astage, A/G	captura WGC/DDA → publica c_seq.

Tabla por módulo II

flow	run_flow; NvofaProvider, GmePipe, MvSmoothPipe, MedianPipe; gme_fit_affine, nvofa_*, gme_*, object_repair, mem_advect/merge/refresh, kGenRing	core; framework (OpticalFlowPipeline) warp_blend (stats); instrument	lee c_seq/c_slots, representing (back-edge); es- cribe f_seq, f_pair_*, bridges hostMV/hostDIS...	Thread F: flujo óptico + GME + holón → publica pares vía f_seq.
warp_blend	WapPipe (wap_create), FieldPipe (fpipe_*), FillPipe (fillpipe_*); matte_mass_count, gme_dispct_from_mask	core (device.hpp), vulkan.h	ninguno (biblioteca stateless de fábricas)	Fábricas de pipelines warp/blend/quality; el <i>dispatch</i> vive en run_present.
present	run_present; UpPipe (up_create), OverlayFpsPC, BridgeSlot; wap_upload, wap_warp_present,	core; framework (PresentSurface, hal); warp_blend; flow; instrument	lee f_seq/f_pair_*/c_seq escribe total_frames, present_cost_us; bridge_img/wapPipeA/A.qT	Thread P: se- lecciona fase, warp/upscale/blit → PresentSurface.submit.

Tabla por módulo III

instrument	<code>dump_bmp, dump_rgba</code>	<code>solo</code> <code>instrument.hpp</code>	<code>ninguno</code>	Volcado de frames (BMP/RGBA), sin acoplamiento a pipeline/FgContext.
framework	<code>PresentSurface</code> (<code>create/submit/device_vulkan</code>), <code>h,</code> <code>OpticalFlowPipeline</code> <code>Win32/D3D11/DXGL/D6comp</code> ivas que lo (<code>init/record_optical_flow/record_warp_code</code>), <code>Error/ErrorCode,</code> <code>optical_flow_*</code> <code>HardwareTopology,</code> <code>hal</code> (<code>f16,</code> <code>cpu_wait,</code> <code>atomics,</code> <code>TSC,</code> <code>SIMD</code>), <code>schema</code>	<code>externo:</code>	<code>N/A</code> (<code>FgContext</code> es <code>app-local</code> ; provee primitivas que lo <code>code</code>),	Substrato ven- dido: <code>P-stage</code> + <code>F-engine</code> + <code>hal/topology/schema</code> .
shaders	<code>*_spv.hpp:</code> <code>hdr_convert,</code> <code>igpu_convert_pack,</code> <code>unpack_packed,</code> <code>igpu_field,</code> <code>wap_warp,</code> <code>wap_fill,</code> <code>mv_smooth,</code> <code>mv_median,</code> <code>nvofa_convert,</code> <code>gme_reduce/solve/dissidence,</code>	<code>nada</code> (bytecode); <code>incluido</code> por <code>core/main.cpp</code>	<code>ninguno</code>	Bytecode SPIR-V de los pipelines compute.

Qué agrega qué, en qué capa (símbolos + línea, structs, shaders) |

Capa	Símbolos nuevos)	clave (archivo:línea	Pipelines/structs propios	Shaders usados
cli	Config@cli.hpp:15; parse_args@cli.cpp:115; parse_extra@cli.cpp:124; print_help@cli.cpp:8; kMaxInterp@cli.hpp:853		Config POD + — enums	
core	FgContext@fg_context.hpp:37; RealSlot@:35; main@main.cpp:167 (ctx@2549, spawn@2797– 2799, join@2806–2809); VDev@device.hpp:20; vdev_create@device.cpp:9; route_for@vk_util.cpp:7; vk_live@globals.cpp:13		VDev/Img/HBuf, FgContext	embebe TODOS los *_spv (main.cpp)
capture	run_capture@capture.cpp:120; d3d_init@:13; cpipe_create@:69; unpipe_create@:93		D3D, ConvPackPipe, UnpackPipe	igpu_convert_pack, unpack_packed, hdr_convert
flow	run_flow@flow.cpp:481; gme_fit_affine@:49; nvofa_create@:166; gme_record@:404;		NvofaProvider, GmePipe, MvSmoothPipe, MedianPipe	nvofa_convert, gme_reduce/solve/dissidence, mv_smooth, mv_median

Qué agrega qué, en qué capa (símbolos + línea, structs, shaders) II

warp_blend	warp_create@warp_blend.cpp:153; fpipe_create@:37; fillpipe_create@:61; matte_mass_count@:14	WapPipe, FieldPipe, FillPipe	wap_warp, wap_fill, igpu_field
present	run_present@present.cpp:57; wap_upload@:694; wap_warp_present@:860; bridge_present_src@:629; paced_wait_P@:509; present@:1530; up_create@:34	UpPipe, BridgeSlot loop	upscale_bilinear/lanczos, unpack_packed, overlay_fps
instrument	dump_bmp@instrument.cpp:11; dump_rgba@:32	—	—
framework	PresentSurface@:139 (create@:145 / submit@:159); OpticalFlowPipeline@:34 (init@:163 / record_optical_flow@:217); Error@:58; HardwareTopology@:111 (probe@:128); f16_to_f32@Simd.hpp:195	PresentSurface, OpticalFlowPipeline	optical_flow_{pyr_down,hier_match,

Conexiones por elemento (si toco X, qué se ve afectado) I

Elemento	Depende de	Lo usan (ripple)
FgContext	tipos VDev/Img/HBuf (core), Config (cli); refs a locales de main()	los 3 threads (run_capture/run_flow/run_present aliasen sus campos); main aggregate-init @2549. Tocar un campo ⇒ re-mira C, F y P.
run_capture	D3D/WgcCtx, VDev A/G, Astage, cfg, img_barrier/full_bic/submit_wait, MMCSS/pin	run_flow (consume c_seq + el anillo); main (spawn)
run_flow	FgContext, OpticalFlowPipeline (ofp/ofpA), NvofaProvider, GmePipe, MvSmoothPipe, MedianPipe, gme_fit_affine, ra::decode_f16, stats de warp_blend	run_present (consume f_seq + f_pair_*); lee p_presenting de P; main (spawn)
run_present	FgContext, PresentSurface, WapPipe (wapPipeA), UpPipe, MedianPipe, hal::cpu_wait_for_ns, TelemetryCsv	panel/DComp (salida); run_flow lee p_presenting (back-edge); main (spawn)
OpticalFlowPipeline	hal, vulkan.h, SPIR-V, optical_flow_*	run_flow (ofp/ofpA: record_optical_flow / record_warp_only)

Conexiones por elemento (si toco X, qué se ve afectado) II

WapPipe

VDev, SPIR-V wap_warp, 14 bindings + run_present (wap_upload +
push 212B (53 floats) wap_warp_present)

Fábricas pipelines

de VDev + llamadas vk*Create* + SPIR-V main las construye en init; los threads
(vdev/img/hbuf/cpipe/unpipe/wap/fpipe/spdpipe/toppipe/avsm/med/refa)
después de la creación de la fábrica de pipelines mira init + el thread dueño.

Para tocar X, ve a Y:

- ▶ Un flag CLI → `cli/cli.cpp` (parse_args / parse_extra) + `Config@cli.hpp:15`
- ▶ El contrato C/F/P → `core/fg_context.hpp` (FgContext) — re-mira los **3 threads**
- ▶ La captura → `capture/capture.cpp` (run_capture)
- ▶ El flujo/holón → `flow/flow.cpp` (run_flow)
- ▶ El warp/blend → `warp_blend` (fábrica) + `present/present.cpp` (dispatch real)
- ▶ El present/pacing → `present/present.cpp` (run_present)

Las flechas del DAG = qué se ve afectado: si tocas un nodo, lo que *apunta hacia él* (sus dependientes) puede romperse. **FgContext** es el hub — un cambio ahí toca C, F y P a la vez. Las fábricas son stateless: cambiar una firma re-mira el init de main más su thread dueño.